

Capitolo 28 — PROC-012 — WCM Change Propagation & Closure

Pubblico	Lettori tecnici e non tecnici del WCM Process & Memory Book
Versione	FROZEN-28
Data master	2026-08-30
Stato	FROZEN
Source path	wcm/documentation/process-memory-book/ chapters/28_proc_012_wcm_change_propagation_closure.md
Source SHA	ef57375
Release	2026-09-04T09:44:50.631Z

GitHub main è la source of truth. Questo PDF è una release derivata: non costituisce approvazione né autorità.

Parte VI — Il Libro dei Processi WCM Stato: FROZEN **Data:** 2026-08-30 **Scope:** WCM generale, domain-agnostic

28.0 Fare una modifica non significa averla davvero chiusa

Quando un sistema cresce, una modifica raramente vive in un solo punto.

Una nuova regola può essere stata implementata correttamente nel luogo in cui nasce e, nello stesso momento, essere ancora assente da un indice, da una documentazione, da un registro, da una relazione di conoscenza o da un controllo automatico che dipende da quella regola.

Per una persona che osserva soltanto il punto modificato, il lavoro può sembrare finito. Per il sistema nel suo insieme, invece, la modifica può essere ancora incompleta.

PROC-012 — WCM Change Propagation & Closure esiste per governare proprio questo problema.

Il principio fondamentale è:

IMPLEMENTED ≠ PROPAGATED ≠ CLOSED

Sono tre stati concettualmente diversi.

- **Implemented** significa che la modifica autorizzata è stata applicata nel suo punto principale.
- **Propagated** significa che tutti gli elementi che devono riflettere quella modifica sono stati aggiornati nel perimetro dichiarato.
- **Closed** significa che la propagazione è stata verificata e il gate di chiusura ha dato esito positivo.

Questo capitolo riguarda le modifiche classificate come **WCM CHANGE**. Non riguarda il normale avanzamento di un'attività già autorizzata come WCM RUN.

28.1 Il problema che PROC-012 risolve

Immaginiamo un esempio pedagogico e astratto.

Un'organizzazione modifica una regola interna: da oggi una certa decisione deve essere approvata prima di una consegna. La regola viene scritta correttamente nel documento principale. Tuttavia:

- il manuale operativo continua a descrivere il comportamento precedente;
- l'indice non rimanda alla nuova regola;
- un controllo automatico non è stato aggiornato;
- un registro continua a mostrare la versione vecchia;
- una vista per gli utenti non riflette il nuovo gate.

La modifica esiste, ma il sistema racconta ancora più verità concorrenti.

Questo è il tipo di incoerenza che PROC-012 vuole impedire.

Il processo non decide se la nuova regola sia giusta o sbagliata. Quella decisione appartiene all'autorità competente. PROC-012 interviene **dopo** che una WCM CHANGE ha ricevuto l'autorità necessaria ed è stata implementata.

La sua domanda è diversa:

la modifica autorizzata è stata propagata in modo completo nel perimetro che essa stessa dichiara di avere toccato?

28.2 Che cos'è una WCM CHANGE nel contesto di questo processo

Nel WCM una modifica materiale a metodo, governance, baseline o canone non viene trattata come un semplice lavoro ordinario.

Prima dell'implementazione esiste un Change Gate:

```
WCM CHANGE
→ BOOTSTRAP / EVIDENCE
→ IMPACT PREVIEW
→ STOP
→ AUTHORITY ESPLICITA DELL'OWNER
→ IMPLEMENTAZIONE CONTROLLATA
```

PROC-012 non sostituisce questo gate e non crea authority.

È importante distinguere i due momenti:

```
PRIMA DELLA MODIFICA
→ devo essere autorizzato a cambiare

DOPO LA MODIFICA
→ devo dimostrare che il cambiamento è stato propagato e può essere chiuso
```

La closure non sana retroattivamente una modifica priva di authority.

Allo stesso modo, un controllo tecnico riuscito non equivale a un'approvazione del contenuto.

28.3 Trigger: quando parte PROC-012

Il trigger è una **WCM CHANGE materiale già autorizzata** che ha raggiunto la fase di implementazione e deve essere portata a chiusura.

Il processo diventa rilevante quando esiste almeno una modifica che può avere impatto su elementi come:

- Agent Start;
- Architecture;
- Process Book;
- Method KB o altri elementi canonici;
- indici e registri;
- documentazione;
- automazioni o flow catalog;
- project layer, quando pertinente;
- Knowledge Assurance o Method Health.

Non significa che ogni change debba modificare tutte queste categorie.

Significa che, per poter chiudere correttamente, il sistema deve dichiarare quali categorie sono impattate e quali no, con una motivazione coerente.

28.4 Input: che cosa serve per poter ragionare sulla chiusura

PROC-012 lavora su una modifica già definita. Gli input principali sono quindi:

1. **identità della WCM CHANGE;**
2. **authority** che ne ha consentito l'implementazione;
3. **baseline precedente** da cui la change è partita;
4. **insieme delle modifiche effettivamente applicate;**
5. **Impact Set dichiarato;**
6. **Change Impact Manifest;**
7. **evidenze di propagazione;**
8. **stato di Knowledge / Method Health**, quando richiesto dal gate finale.

Il processo non deve ricostruire questi elementi per intuizione quando esistono già in forma strutturata.

Se manca un'informazione necessaria alla verifica, il comportamento corretto è fail closed, non colmare il vuoto con un'ipotesi.

28.5 Il Change Impact Manifest: la mappa dichiarata della propagazione

Il cuore operativo di PROC-012 è il **Change Impact Manifest**.

Nella baseline corrente ogni WCM CHANGE materiale usa un manifest strutturato sotto:

`wcm/change-manifests/<change-id>.json`

Il template di riferimento è:

`wcm/process-book/templates/WCM_CHANGE_IMPACT_MANIFEST_TEMPLATE.json`

Il manifest non è un riassunto narrativo della modifica. È una dichiarazione strutturata di ciò che deve essere verificato prima della chiusura.

Contiene almeno:

- change ID e titolo;
- authority;
- stato previsto per l'ingresso nel gate di chiusura;
- impatto **YES/NO** per le categorie rilevanti;
- motivazione dell'impatto;
- file coinvolti;
- indici o registri che devono essere coerenti;
- eventuali note di scope.

Il suo valore è rendere controllabile una domanda che altrimenti resterebbe vaga:

| *“abbiamo aggiornato tutto ciò che questa modifica richiedeva?”*

Con il manifest, la domanda diventa verificabile rispetto a un perimetro dichiarato.

28.6 Impact Set: non aggiornare tutto, aggiornare ciò che è realmente impattato

La propagazione non significa modificare indiscriminatamente l'intera memoria WCM.

Significa identificare l'**Impact Set** corretto.

Un esempio pedagogico: se cambia una regola che riguarda soltanto un protocollo e il relativo indice, non avrebbe senso riscrivere documenti non collegati solo per dimostrare attività.

All'opposto, se una modifica cambia un protocollo e anche il comportamento di un'automazione che lo implementa, aggiornare soltanto il protocollo sarebbe insufficiente.

La disciplina è quindi:

```
CHANGE
→ IDENTIFICA IMPATTO REALE
→ DICHIARA IMPACT SET
→ PROPAGA SOLO DOVE NECESSARIO
→ VERIFICA COVERAGE
```

La completezza non coincide con il numero di file modificati.

Coincide con la copertura corretta delle conseguenze materiali della change.

28.7 La sequenza canonica

La baseline corrente descrive la sequenza così:

```
WCM CHANGE
→ Impact Preview
→ explicit owner authority
→ controlled implementation
→ Change Impact Manifest
→ propagation to declared Impact Set
→ deterministic propagation check
→ Knowledge / Method Health check
→ PASS
→ CLOSED
```

Ogni passaggio svolge una funzione diversa.

Impact Preview

Serve prima della scrittura. Rende visibili effetti e rischi della modifica proposta.

Explicit owner authority

Autorizza quella specifica change dopo il preview. Non può essere sostituita da un consenso precedente e generico.

Controlled implementation

Applica la modifica senza ampliare autonomamente scope o authority.

Change Impact Manifest

Formalizza che cosa deve essere propagato e verificato.

Propagation

Aggiorna gli elementi dichiarati nell'Impact Set.

Deterministic propagation check

Verifica copertura, presenza dei path, registri e invarianti strutturali previsti.

Knowledge / Method Health check

Verifica che il sistema non presenti condizioni di salute incompatibili con la closure quando questo controllo è richiesto.

Closure

Arriva soltanto dopo il PASS del gate.

28.8 Gate di chiusura: perché esiste

Senza un gate finale, la frase “abbiamo finito” rischia di dipendere dalla percezione di chi ha eseguito la modifica.

PROC-012 sostituisce questa percezione con condizioni verificabili.

La baseline richiede che il checker fallisca, tra gli altri casi, quando:

- manca il Change Impact Manifest;
- una categoria realmente impattata non è dichiarata come richiesta;
- un path dichiarato non esiste;
- manca un indice o registro obbligatorio per una categoria modificata;
- un file canonico cambiato non è coperto dal manifest;
- cambia un flow o workflow ma l'Automation Catalog non è incluso quando pertinente;
- cambiano processi o protocolli ma il Process Register non è coperto;
- cambia l'Architecture ma il relativo index non è coperto;
- cambia Method KB/canon ma manca il relativo index;
- cambia la documentazione ma il Documentation Index non è incluso;
- la closure richiede Method Health e questo non risulta **HEALTHY**.

Questi controlli non stabiliscono la correttezza semantica della modifica.

Stabiliscono se la propagazione dichiarata possiede la copertura strutturale necessaria.

28.9 Determinismo senza autorità semantica

Una caratteristica essenziale di PROC-012 è la separazione tra significato e verifica.

Wise / Cognitive Core può determinare, entro l'authority disponibile, il significato dell'Impact Set e produrre i contenuti semantici autorizzati.

Il **deterministic checker** può verificare condizioni come:

- file presente o assente;
- path valido o invalido;
- categoria dichiarata o non dichiarata;
- index richiesto coperto o non coperto;
- file cambiato incluso o escluso dal manifest;
- Method Health conforme o non conforme al requisito.

Il checker non può concludere autonomamente:

| *“questa nuova regola è migliore, quindi la considero approvata”.*

Questo sarebbe un salto di authority.

La distinzione da ricordare è:

SEMANTIC CORRECTNESS
≠
STRUCTURAL CLOSURE VALIDATION

PROC-012 usa il secondo tipo di controllo per proteggere il primo, non per sostituirlo.

28.10 Il problema delle change distribuite su più commit

Una modifica materiale può richiedere più passaggi tecnici.

Se il controllo osservasse soltanto l'ultimo commit, potrebbe perdere file modificati nei commit precedenti della stessa change.

Per questo la baseline corrente usa un intervallo completo di closure.

Il Change Impact Manifest schema 1.1 dichiara un `base_sha`: il commit esatto immediatamente precedente al primo commit della WCM CHANGE.

Il checker verifica quindi l'intervallo:

```
base_sha..closure_head_sha
```

In termini semplici, non guarda soltanto “l'ultima fotografia”.

Guarda tutto il tratto di storia che appartiene alla change.

Questo permette di controllare che ogni file canonico modificato lungo l'intero intervallo sia coperto dal manifest.

Se il `base_sha` manca, è malformato, non è raggiungibile o non appartiene correttamente alla storia del closure head, il gate fallisce.

28.11 Perché l'exact head conta

Un branch può continuare a ricevere nuovi commit mentre un controllo è in esecuzione.

Se la closure verificasse genericamente “lo stato attuale di main”, il perimetro potrebbe cambiare tra l'inizio e la fine del controllo.

PROC-012 evita questa ambiguità legando la closure a un head esatto.

```
CHANGE RANGE ESATTO
+
CLOSURE HEAD ESATTO
→ VERIFICA RIPRODUCIBILE
```

Il principio è simile a controllare una versione numerata di un documento invece di dire semplicemente “controlla l'ultima”.

L'esattezza dell'identità non serve a complicare il processo: serve a rendere ripetibile ciò che viene dichiarato chiuso.

28.12 READY_FOR_CLOSURE non significa CLOSED

Il manifest descrive una change pronta a entrare nel gate.

Per questo può avere stato **READY_FOR_CLOSURE**.

Ma essere pronti per il controllo non equivale ad averlo superato.

```
READY_FOR_CLOSURE
→ PROPAGATION GATE
→ PASS?
  └─ NO → NOT CLOSED
  └─ YES → CLOSURE EVIDENCE
```

Questa distinzione protegge il sistema da una scorciatoia pericolosa: trattare l'intenzione di chiudere come prova della chiusura.

28.13 Closure Receipt: rendere durevole l'esito del gate

Nella baseline corrente, un PASS di closure non resta soltanto un risultato temporaneo di una pipeline tecnica.

Viene materializzato attraverso una **Closure Receipt** immutabile sotto:

wcm/change-manifests/results/<change_id>.json

La receipt descrive l'esito del gate.

Il manifest e la receipt svolgono ruoli diversi:

```
CHANGE IMPACT MANIFEST
= ciò che entra nel gate

CLOSURE RECEIPT
= risultato verificato del gate
```

Il writer della receipt è progettato per essere idempotente e fail closed.

Se esiste già una receipt coerente, il replay non deve creare duplicati.

Se esiste una receipt con core incompatibile, il sistema deve fermarsi invece di sovrascriverla opportunisticamente.

Anche qui la receipt non crea authority: conserva l'evidenza dell'esito di controlli già autorizzati.

28.14 Method Health come condizione di closure

Una change può avere propagato i file previsti e tuttavia lasciare il sistema in uno stato di conoscenza non sano.

Per il percorso di closure finale previsto dalla baseline corrente, Method Knowledge Health deve essere coerente con il requisito del gate.

Quando il gate richiede **HEALTHY**, uno stato diverso impedisce la closure.

Questo evita una situazione paradossale:


```
FILE PRESENTI
+
INDICI PRESENTI
+
KNOWLEDGE HEALTH NON VERDE
→ NON CLOSED
```

Il punto non è pretendere perfezione astratta.

È impedire che una change venga dichiarata chiusa mentre i controlli di salute previsti dal processo indicano ancora un problema materiale.

28.15 Historical Closure Backfill: recuperare change storiche senza inventare prove

La baseline include anche un meccanismo di recovery per WCM CHANGE storiche rimaste in uno stato equivalente a **READY_FOR_CLOSURE**.

Il principio è conservativo.

Una change storica può essere chiusa retrospettivamente soltanto se esistono abbastanza evidenze per ricostruire in modo esplicito il perimetro da verificare.

Per manifest moderni, il controllo usa il **base_sha** canonico.

Per manifest legacy, il recovery richiede un **legacy_base_sha** esplicito e applica un validator più restrittivo.

Non è ammessa una discovery fuzzy del range.

```
EVIDENCE SUFFICIENTE
→ CHECK
→ POSSIBILE RECEIPT

EVIDENCE INSUFFICIENTE
→ FAIL CLOSED
→ NO RECEIPT
```

Il recovery non deve trasformarsi in una sanatoria basata sulla probabilità.

28.16 Failure mode principali

PROC-012 tratta come failure, tra gli altri, i seguenti casi:

- manifest mancante;
- manifest strutturalmente invalido;
- Impact Set incompleto;
- file canonico modificato ma non coperto;
- indice o registro obbligatorio non incluso;
- path dichiarato inesistente;
- range multi-commit non valido;
- base SHA non ancestor del closure head;

- manifest fuori dal range dichiarato;
- Method Health non conforme quando richiesto;
- evidence insufficiente nel backfill storico;
- collisione o incoerenza nella Closure Receipt.

La risposta corretta non è degradare silenziosamente il gate.

È:

```
DETECT
→ FAIL CLOSED
→ PRESERVE AUTHORITY
→ REPAIR / COMPLETE PROPAGATION
→ RERUN GATE
```

Una WCM CHANGE che fallisce il gate può essere implementata, ma resta **NOT CLOSED**.

28.17 Relazioni con altri elementi WCM

PROC-012 non opera isolatamente.

Dipende in particolare da:

- **PROC-006 – Memory Consolidation & Consistency Loop**, perché una modifica materiale deve consolidare correttamente la memoria persistente;
- **PROC-008 – Knowledge Integrity Assurance Loop**, per la verifica dell'integrità della conoscenza;
- **PROC-010 – Documentation Continuity Loop**, quando esiste impatto documentale;
- **PROT-019 – WCM Change Closure Standard**, che vincola la disciplina di closure;
- **PROT-015 – Documentation Impact & Publication Standard**, quando la change ha conseguenze documentali;
- **PROT-017 – Persistent Mutation Safety**, per le mutazioni persistenti applicabili.

La relazione può essere letta così:

```
CHANGE AUTORIZZATA
→ IMPLEMENTAZIONE
→ CONSOLIDATION / ASSURANCE / DOCUMENTATION IMPACT
→ PROPAGATION CHECK
→ CLOSURE
```

PROC-012 è quindi un processo di **chiusura sistemica**, non un sostituto degli altri loop.

28.18 Gate e decision point

I principali decision point del processo sono:

1. La change è materiale?

Se non lo è, PROC-012 può non essere applicabile come closure di WCM CHANGE materiale.

2. Esiste authority valida?

Se manca, la closure non può sanare il problema.

3. L'Impact Set è completo?

Se no, la propagazione deve essere completata prima di procedere.

4. Il manifest copre i changed file rilevanti?

Se no, fail closed.

5. Gli indici e registri obbligatori sono coperti?

Se no, fail closed.

6. Il Method Health soddisfa il requisito del gate?

Se no, la change resta not closed.

7. Il gate è PASS?

Solo in questo caso può essere prodotta l'evidence di closure prevista.

28.19 Output

Gli output attesi del processo sono, nel perimetro applicabile:

- Change Impact Manifest valido;
- propagazione completata sul dichiarato Impact Set;
- esito deterministico del propagation check;
- esito Knowledge / Method Health richiesto;
- Closure Receipt quando il gate finale è PASS;
- stato effettivo della WCM CHANGE coerente con l'esito.

Il risultato più importante non è un singolo file.

È la possibilità di affermare, con evidence strutturata, che:

```
LA MODIFICA AUTORIZZATA  
È STATA PROPAGATA  
NEL PERIMETRO DICHIARATO  
E IL GATE DI CHIUSURA È PASS
```

28.20 Maturity e limiti

Il processo canonico è **ACTIVE / FIRST FIELD VALIDATION**.

Questa qualificazione è importante.

Significa che PROC-012 appartiene alla baseline operativa corrente e possiede implementazioni ed evidence iniziali, ma non autorizza a sostenere che ogni possibile tipo di WCM CHANGE, repository, topology organizzativa o scenario futuro sia già stato validato universalmente.

La baseline include:

- Change Impact Manifest;
- checker deterministico;
- controllo event-driven;
- full-range multi-commit closure;
- exact head;
- Method Health gate;
- Closure Receipt persistente;
- recovery/backfill storico con vincoli fail-closed.

Restano però validi i limiti generali del WCM:

- la completezza semantica dell'Impact Set richiede cognition e authority appropriata;
 - il determinismo verifica invarianti già formalizzati, non inventa significato;
 - una nuova categoria d'impatto non può essere aggiunta implicitamente dal testo editoriale;
 - l'evidence di field validation va letta nel perimetro osservato, non generalizzata oltre misura.
-

28.21 Un esempio pedagogico completo

Consideriamo un esempio astratto.

Viene autorizzata una change che modifica il modo in cui una richiesta deve essere approvata.

Dopo l'implementazione si scopre che la modifica tocca:

- una regola canonica;
- il relativo registro;
- un documento che spiega il flusso;
- un controllo automatico.

Il manifest dichiara queste quattro aree come impattate.

La propagazione aggiorna i file necessari.

Il checker verifica che:

- i path esistano;
- il file canonico modificato sia coperto;
- il registro sia incluso;
- il documento sia incluso;
- il controllo automatico rientri nel perimetro dichiarato;
- il Method Health richiesto sia verde.

Se tutto passa, viene prodotta la Closure Receipt e la change può essere considerata chiusa.

Se manca anche soltanto il registro, la change resta implementata ma non chiusa.

Questo esempio è soltanto pedagogico: serve a mostrare la logica del processo e non introduce una nuova regola WCM.

28.22 La regola finale da ricordare

PROC-012 protegge il WCM da una delle forme più comuni di incoerenza nei sistemi complessi: una modifica corretta localmente ma incompleta globalmente.

La sua disciplina può essere ricordata così:

```
NON CHIUDERE QUANDO HAI FINITO DI MODIFICARE.  
CHIUDI QUANDO HAI DIMOSTRATO DI AVERE PROPAGATO.
```

Oppure, nella forma canonica più sintetica:

```
IMPLEMENTED ≠ PROPAGATED ≠ CLOSED
```

Una WCM CHANGE è realmente chiusa soltanto quando il suo perimetro è stato dichiarato, propagato e verificato dal gate previsto, senza ampliare authority e senza sostituire la verifica con un'impressione di completezza.

Source Map

Fonte canonica primaria

- [wcm/process-book/processes/PROC-012_WCM_CHANGE_PROPAGATION_CLOSURE.md](#)

Fonti collegate richiamate dal processo

- [wcm/process-book/templates/WCM_CHANGE_IMPACT_MANIFEST_TEMPLATE.json](#)
- [wcm/runtime/change_propagation_check.py](#)
- [.github/workflows/wcm-change-propagation.yml](#)
- [PROC-006 – Memory Consolidation & Consistency Loop](#)
- [PROC-008 – Knowledge Integrity Assurance Loop](#)
- [PROC-010 – Documentation Continuity Loop](#)
- [PROT-015 – Documentation Impact & Publication Standard](#)
- [PROT-017 – Persistent Mutation Safety](#)
- [PROT-019 – WCM Change Closure Standard](#)

Qualificatori di verità e maturity

- processo canonico: [ACTIVE](#) / [FIRST FIELD VALIDATION](#);

- nessun claim di FIELD VALIDATION universale;
- nessun claim di originalità assoluta;
- closure deterministica limitata agli invarianti formalizzati;
- authority della WCM CHANGE resta dell'owner e non viene ampliata dal closure gate;
- esempi del capitolo: esclusivamente pedagogici e domain-agnostic, non nuove regole WCM.